

tf.compat.v1.data.experimental.make_csv_dataset

Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/data/experimental/make_csv_dataset

Reads CSV files into a dataset.

Code Examples

```
tf.compat.v1.data.experimental.make_csv_dataset(  
    file_pattern,  
    batch_size,  
    column_names=None,  
    column_defaults=None,  
    label_name=None,  
    select_columns=None,  
    field_delim=',',  
    use_quote_delim=True,  
    na_value='',  
    header=True,  
    num_epochs=None,  
    shuffle=True,  
    shuffle_buffer_size=10000,  
    shuffle_seed=None,  
    prefetch_buffer_size=None,  
    num_parallel_reads=None,  
    sloppy=False,  
    num_rows_for_inference=100,  
    compression_type=None,  
    ignore_errors=False,  
    encoding='utf-8'  
)
```

```
tf.compat.v1.data.experimental.make_csv_dataset(
file_pattern,
batch_size,
column_names=None,
column_defaults=None,
label_name=None,
select_columns=None,
field_delim=',',
use_quote_delim=True,
na_value='',
header=True,
num_epochs=None,
shuffle=True,
shuffle_buffer_size=10000,
shuffle_seed=None,
prefetch_buffer_size=None,
num_parallel_reads=None,
sloppy=False,
num_rows_for_inference=100,
compression_type=None,
ignore_errors=False,
encoding='utf-8'
)
```

```
# No label column specified
dataset = tf.data.experimental.make_csv_dataset(filename,
batch_size=2)
iterator = dataset.as_numpy_iterator()
print(dict(next(iterator)))
# prints a dictionary of batched features:
# OrderedDict([('Feature_A', array([1, 4], dtype=int32)),
#              ('Feature_B', array([b'a', b'd'], dtype=object))])
```

```
# No label column specified
dataset = tf.data.experimental.make_csv_dataset(filename,
batch_size=2)
iterator = dataset.as_numpy_iterator()
print(dict(next(iterator)))
# prints a dictionary of batched features:
# OrderedDict([('Feature_A', array([1, 4], dtype=int32)),
#              ('Feature_B', array([b'a', b'd'], dtype=object))])
```

```
# Set Feature_B as label column
dataset = tf.data.experimental.make_csv_dataset(
filename, batch_size=2, label_name="Feature_B")
iterator = dataset.as_numpy_iterator()
print(next(iterator))
# prints (features, labels) tuple:
# (OrderedDict([('Feature_A', array([1, 2], dtype=int32))]),
#  array([b'a', b'b'], dtype=object))
```

```
# Set Feature_B as label column
dataset = tf.data.experimental.make_csv_dataset(
filename, batch_size=2, label_name="Feature_B")
iterator = dataset.as_numpy_iterator()
print(next(iterator))
# prints (features, labels) tuple:
# (OrderedDict([('Feature_A', array([1, 2], dtype=int32))]),
#  array([b'a', b'b'], dtype=object))
```

Documentation

Reads CSV files into a dataset.

Reads CSV files into a dataset, where each element of the dataset is a (features, labels) tuple that corresponds to a batch of CSV rows. The features dictionary maps feature column names to Tensors containing the corresponding feature data, and labels is a Tensor containing the batch's label data.

By default, the first rows of the CSV files are expected to be headers listing the column names. If the first rows are not headers, set `header=False` and provide the column names with the `column_names` argument.

By default, the dataset is repeated indefinitely, reshuffling the order each time. This behavior can be modified by setting the `num_epochs` and `shuffle` arguments.

For example, suppose you have a CSV file containing

See the
Load CSV data guide for
more examples of using `make_csv_dataset` to read CSV data.

Returns

Raises